Verified certificate refinements for K215

Balanced source checking, a reusable marker theorem,
and concrete zero-credit enumeration

Research and integration packet for Vladimir Reshetnikov's ProveIt

21 October 2026

Abstract

We inspect the current 215-state ZFC-consistency machines, the existing Lean and Rocq correctness developments, and all seven proposal archives in `docs/incoming`. Three further formal contributions are supplied. First, a balanced, binary-indexed certificate checker proves the exact K215 source equivalence for every register file, with the old checker as its verified specification. Second, one finite-table checker proves arbitrary-tape halting equivalence for the M124/FM215 marker family, including the explicitly instantiated provider-125 member. Third, the actual Cantor command encoding and exhaustive schedule satisfy the zero-credit prefix theorem, with no abstract parsing or scheduler hypotheses. All new formal contributions were compiled using Coq 8.18.0, with closed theorem assumption reports; the precise recursive-check scopes are recorded. A wider local-fold search found no successful reduction to 214 states. We do not claim a 214-state machine, a state-count lower bound, or a fresh rebuild of the complete inherited consistency endpoint. The report distinguishes existing proposals, new verified refinements, finite corroboration, and remaining integration work.

Repository pin. `53269bcab1efc0f82d3022645bd7cf0c22d84959`.
Machine convention. One two-way infinite binary tape, initially zero; head and initial control at 0; L/R moves only; 215 working states, with a separate halt excluded from the count. Initialization is included.

Contents

1 Results and recommendations **3**
1.1 Direct answers . . . 3
1.2 The existing formal baseline . . . 3
1.3 What the incoming proposals already establish . . . 4
1.4 Evidence levels in this packet . . . 5

2 Verified balanced source certificates **5**
2.1 Removing a representation cost . . . 5
2.2 Exact K215 theorem . . . 6
2.3 Measured work and build-time limits . . . 6

3 One finite checker for a family of marker variants **7**
3.1 The family already present in the proposals . . . 7
3.2 Generic rule and finite obligations . . . 8
3.3 Concrete instances and M125 table . . . 9
3.4 Fresh verification and precise scope . . . 9

4 A concrete zero-credit theorem for ZC215 **10**
4.1 Why this is a distinct simplification . . . 10
4.2 Exact codec, including zero . . . 10
4.3 Occurrence injection and prefix safety . . . 11
4.4 Application and remaining machine work . . . 12

5 Attempt to remove another state **12**
5.1 What was searched . . . 12
5.2 Controls and relationship to earlier searches . . . 13
5.3 What remains plausible . . . 14

6 Proof architecture and integration plan **14**
6.1 Which proof should be shortened? . . . 14
6.2 Recommended sequence for the current endpoint . . . 15
6.3 Keep four small semantic interfaces . . . 16
6.4 Artifacts and reproduction . . . 16
6.5 Acceptance criteria for a future 214-state claim . . . 17

1 Results and recommendations

1.1 Direct answers

Shortening the existing proofs. Preserve the exact primary K215 and its public endpoint statements. Adopt the already checked semantic reductions from the incoming packets, then replace the ordinary source checker's repeated linear list lookups by the verified balanced representation supplied here. This changes the certificate implementation without changing the source program, numerical calculus, binary machine, or consistency statement. The new theorem is the same all-register source equivalence required by the existing proof. This is a concrete checked refinement, rather than a proposal to trust a faster external verifier.

An easier additional 215-state proof. The two-marker family containing M124 and FM215 is particularly economical when the existing K215 endpoint is reused. Its entire new mathematical argument is a two-clause tape invariant and a one-or-three-step local simulation. We package that argument once as a generic finite checker; three literal instances then occupy 35 lines of Coq, including an explicit calibration-transfer corollary and assumption reports. The already supplied CM215 dense/Core bridge may make CM215 the quickest production integration, even though its marker invariant has one additional case. A short relative proof does not eliminate the inherited K215-to-ZFC dependency closure.

A cleaner independent enumerator proof. ZC215's exhaustive raw-code enumeration admits an especially simple protected-credit proof. We complete the concrete formalization: the actual Cantor encoding, actual decoder, and actual increasing candidate order satisfy the required inequality at every prefix. This is a useful semantic component for a ZC215 formalization; it is not a new complete ZC215 binary endpoint.

214 states. No valid machine with 214 or fewer working states was constructed. The expanded local search and its limitations are documented in Section 5. The explicit M125 table supplied here has 215 working states and instantiates the provider-125 option already mentioned in the earlier M124 note. Neither that instance nor the new proof machinery constitutes a one-state saving.

1.2 The existing formal baseline

The accepted native K215 table has 430 rows and SHA-256

`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2.`

Its dense form uses working labels 0–214 and halt 215. The maintained endpoints are:

Lean: `KEndpoint215.blank_halts_iff_zfc_refutation`
Rocq: `K215F.CorrectDense215.tm215_halts_iff_ZFC_inconsistent`

They express halting iff a finite ZFC refutation exists. Negation gives the consistency equivalence without taking consistency as a premise [1, 2].

The verification records distinguish the original fresh build from subsequent incremental simplification. The original Lean run compiled 204 project modules; the current selected closure has 185. The original Rocq run compiled 237 sources. Phase four rebuilt ten affected Lean modules and reused 175 exactly bound predecessor objects. Its three focused files decreased from 1,814 to 1,606 lines. Those are existing repository results, not fresh measurements from this packet [3].

The formal endpoints are external theorems about syntactic ZFC refutations. The repository explicitly distinguishes them from a proof encoded inside ZFC's own proof syntax or an internal PA endpoint. The present work preserves that distinction. Its finite simulations and arithmetic arguments can be formalized in ZFC at the ordinary mathematical level; it does not deliver a new internal proof-code object or assert unconditional nonhalting.

1.3 What the incoming proposals already establish

All seven archives present at the pinned commit were inspected. Their exact identities are retained in `zero_credit/incoming_inventory.json`. The following reconciliation is essential to avoid repeatedly proposing work that has already been done [4].

Incoming packet (date suffix omitted)	Existing contribution and boundary
`K215_Proof_Simplification`	Checked CM215 transfer; an exact 1,426-entry normalized source certificate and source theorem. No fresh whole-endpoint rebuild.
`K215_Proof_Simplification (2)`	Checked CM215 dense transfer; ordinary zero-offset Lean source lane; compact certificate and Rocq mirror theorem. Final full-ZFC adapter remains uncompiled in that packet.
`K215_Proof_Transport`	Concrete CM215 all-tape and dense-semantics bridges; a final endpoint wrapper is supplied but uncompiled.
`ZFC_K215_Proof_Transport`	CM215 native transfer, positive-stuttering theorem, shared-tree and compact-path pilots; representation integration is separate.
`ZFC215_Proof_Simplification`	M124 Lean/Rocq all-tape transfers; ordinary one-sided source theorem; 88.4601% certificate-source compression. Complete M124 representation bridge remains separate.
`zfc-fm215-proof-compression`	FM215 native all-tape Lean transfer and a 234-role decoder certificate. Full endpoint bridge remains separate.
`Structural_Proof_Simplification`	Normalized source-graph audit, semantic seed round-trip proposal, startup sweeps. New formal proposals are uncompiled; the canonical certificate fails the unchanged raw-pointer checker.

The offset symmetry theorem, ordinary zero-offset source proof, compact path encoding, normalized 1,426 checkpoints, and positive-stuttering transfer are therefore credited as prior work. Balanced lookup implements an opportunity left open in the incoming Rocq report. The new marker checker shares the prior marker argument across a family. The new zero-credit file closes the concrete encoding and scheduling hypotheses of an earlier abstract prototype.

1.4 Evidence levels in this packet

Contribution	Fresh validation	Deliberate boundary
Balanced source theorem	Coq 8.18 compilation, closed assumptions; recursive check of generic checker	No full 237-source endpoint replay; concrete theorem checked by compilation
Marker family	Three modules compiled; six closed reports; recursive check of concrete instances	Native arbitrary-tape model; repository representation bridge separate
Concrete zero credit	408-line file compiled; nine closed reports; recursive check passed	Exact arithmetic scheduler, not physical TM execution
State search	Reproducible C++/Python, controls, exact hashes and counts	Bounded sufficient proof test; no lower bound

All new Coq builds used version 8.18.0 with OCaml 4.14.1. This is different from the maintained Rocq 9.1.0 lane. The balanced package preserves the original 9.1 source and its proved compatibility shim. No claim is made that this task freshly ran either the full Lean endpoint or the full Rocq 9.1 endpoint. Machine-specific proof-assistant results are kept distinct from Python checks of exact rows and bounded startup runs.

2 Verified balanced source certificates

2.1 Removing a representation cost

The ordinary source certificate is a list of checkpoint entries (P_i, Q_i, j_i, k_i). The paths P_i, Q_i name corresponding source sites; j_i, k_i name the paired continuations of a decrement test. The existing checker retrieves a continuation with `nth_error C (N.to_nat j)`. A single access can traverse a linear prefix of C, even when its index is already represented in binary. Repeated accesses create a quadratic worst-case indexing component.

The program semantics do not require a list. Nor does the soundness proof need to trust an external random-access implementation. We replace storage by a tree and prove equality with the original list lookup. This lets the existing soundness theorem remain the specification. The incoming ordinary checker and mirror theorem provide the semantic foundation [4].

Definition 2.1 (Certificate tree)**.** A certificate tree is empty, a leaf containing one checkpoint entry, or a node $\mathrm{Node}(c, L, R)$ with binary natural cut c. Its flattening is the list of leaves in left-to-right order. Validity means that every node's cut equals the number of leaves in its left subtree, and that both subtrees are valid.

Lookup at j follows L if $j < c$, otherwise R at $j - c$. A leaf succeeds only for index zero; an empty tree returns no entry. Binary `N` values are used for cuts and indices throughout lookup. In particular, invalid indices retain the original failure behavior.

Theorem 2.2 (Lookup refinement)**.** *For every valid certificate tree T and every $j \in \mathbb{N}$,*

$$\mathrm{lookup}(T, j) = \mathrm{nth_error}(\mathrm{flatten}(T), \mathrm{toNat}(j)).$$

Proof. Induct on T. Empty and singleton lists give the two base cases, including out-of-range failure. At a node, validity identifies the cut with the left list length. If $j < c$, the list-append lookup law selects that left list and the induction hypothesis applies. If $j \geq c$, the other append lookup law selects the right list at the subtracted index. The binary-to-natural subtraction identity supplies exactly the original index. No assumption that j is in range is used. □

Theorem 2.3 (Checker refinement)**.** *For valid T and arbitrary source decoder trees t_1, t_2,*

$$\mathrm{checkTree}(t_1, t_2, T) = \mathrm{check}(t_1, t_2, \mathrm{flatten}(T)).$$

Proof. Transport Theorem 2.2 through the existing continuation, segment, and entry checks. The tree-wide Boolean fold equals a list-wide `forallb` over its flattening by structural induction. Combining these identities gives the result, including the initial continuation check. The segment interpreter and all register effects are unchanged. □

Consequently a true tree-validity check and a true forward certificate imply the same halting equivalence as the original ordinary bisimulation theorem. The incoming one-sided symmetry theorem supplies the reverse direction; no second expensive mirrored certificate evaluation is introduced.

2.2 Exact K215 theorem

`Cert215Tree.v` constructs a concrete tree from `Plain215`. The kernel checks its validity, exact flattening identity, 2,240 entries, and height 13, counting the leaf. A general structural lemma bounds lookup depth by tree height; the concrete bound therefore holds for every binary index, including invalid indices.

Theorem 2.4 (Compiled exact source equivalence)**.** *For every natural-valued register file R,*

$$\mathrm{rm_halts}(p_{215}, ([], R)) \iff \mathrm{rm_halts}(\mathrm{dagPerm}, ([], R)).$$

The compiled name is `Source215Balanced.p215_dagPerm_balanced`. It uses the exact current programs, not models with the same leaf counts. The byte bindings for `Prog215.v`, `DagPerm.v`, and the original offset certificate are recorded in the package. The new generic module has 194 lines, its exact tree wrapper 23, and the source theorem 21. These are reusable additions; they are not advertised as a reduction in total source lines. Their purpose is a simpler certificate representation with a verified implementation boundary.

The certificate construction itself need not be trusted. Even if a generator or the fuel-based builder produced a wrong tree, the exact flattening, validity, and forward-check theorems would fail to compile. This is a useful division of responsibility: the generator searches or compresses; the kernel checks the resulting finite object against an explicit specification.

2.3 Measured work and build-time limits

The exact certificate contains 2,239 decrement checkpoints and one halt. Its two branch accesses per decrement, plus the initial access, give 4,479 lookups. The independent census gives:

Quantity	List representation	Balanced representation
Traversal nodes visited	5,018,706	54,514
Mean nodes per access	1,120.50	12.17
Maximum balanced depth	—	13

The indexing traversal decreases by a factor of about 92.06. For balanced n-entry trees and $O(n)$ accesses, the node visits are $O(n \log n)$. Charging binary comparison and subtraction bit by bit gives $O(n \log^2 n)$ indexing work for in-range indices. This is an analysis of the indexing component; source decoding, segment execution, proof reconstruction, and kernel checking are additional work.

One matched observation in the shared environment produced:

Measurement	Plain	Balanced
`Time vm_compute` transaction	16.008 s	11.314 s
Complete source-module compiler process	33.103 s	24.772 s

Those are illustrative single observations, approximately 29.3% and 25.2% lower respectively. The new tree-construction/exactness module itself took 13.159 seconds. Thus this experiment does *not* establish a first-build speedup, and it makes no whole-endpoint timing claim. A maintained generator could emit the balanced tree directly and keep exact flattening as a separate audit target; that design has not been measured here.

All three new modules compiled with closed assumption reports. A separate recursive `coqchk` of the generic checker and its dependencies passed. The concrete source theorem was kernel-checked during compilation, but was not the target of that separate recursive invocation. The complete inherited ZFC endpoint was not rebuilt.

3 One finite checker for a family of marker variants

3.1 The family already present in the proposals

M124 and FM215 are closely related members of the same predecessor-marker family. An exact comparison finds that 429 of their 430 rows agree; their only difference is

$$(127, 1): \quad 1R59 \quad \text{versus} \quad 1R109.$$

Both providers satisfy the required zero-return equation:

$$(59, 0) = (109, 0) = (125, 0) = 0R62.$$

The earlier M124 note explicitly permits providers 109 and 125. Accordingly, we do not claim a new marker idea for the provider-125 instance. The new contribution is a generic verified checker that packages the shared argument, and its compiled application to three exact tables [4].

The crucial parent rows are

Old role	Read 0	Read 1
124	$0R62$	$1R138$
127	$0R14$	$1R138$

Every old incoming row to role 124 writes a 1 and moves right; every old incoming row to role 127 writes a 0 and moves right. Thus the bit immediately left of the head distinguishes the two logical roles. The roles can be identified at new control 124, while physical control 127 is reused to read that origin marker.

3.2 Generic rule and finite obligations

Let the two old roles be a, b, with a common one-successor w and zero-successors u, v. Define $\phi(b) = a$ and $\phi(q) = q$ otherwise. The old zero transitions preserve zero and move right to u, v; both old one transitions preserve one and move right to w. Let r_0, r_1 be existing return providers. The new local rows have the following shape:

$$\begin{aligned} (a,0) &\mapsto 0Lb, & (a,1) &\mapsto 1R\phi(w),\\ (b,0) &\mapsto 0Rr_0, & (b,1) &\mapsto 1Rr_1,\\ (r_0,0) &\mapsto 0R\phi(v), & (r_1,0) &\mapsto 0R\phi(u). \end{aligned}$$

Outside a, b, the new table agrees with the old table after mapping source controls and target controls by ϕ.

The Boolean checker verifies these exact shapes, every ordinary row, both incoming marker conditions, equal finite table lengths, distinct noninitial roles, nonidentification with halt, and absence of outgoing halt rows. Its soundness theorem covers indices beyond the stored tables as well: the finite lookup returns `None` there, and equal lengths preserve that behavior. No assumption about a particular startup or source register frame is hidden in the concrete checks.

Here Halt means reaching the designated halt label under the totalized step function, which fixes a configuration on a missing row. The concrete closure checks below identify this predicate with conventional halting.

Theorem 3.1 (Generic checked marker fold)**.** *If* `check O N a b halt u v w r0 r1 = true`*, then for every complete Boolean tape $t : \mathbb{Z} \to \{0,1\}$ and every head position h,*

$$\mathrm{Halt}(O, (0, h, t)) \quad \Longleftrightarrow \quad \mathrm{Halt}(N, (0, h, t)).$$

Proof. The old invariant consists of exactly two clauses:

$$q = a \Longrightarrow t(h-1) = 1, \qquad q = b \Longrightarrow t(h-1) = 0.$$

It holds initially because $a, b \neq 0$. Each incoming-row condition reestablishes its marker after one old step. An ordinary old transition is matched by one new transition under ϕ. At either merged role on read 1, the common one-successor gives the same one-step result. On read 0, the new machine moves left to the selector, reads and preserves the origin marker, moves right to the corresponding provider, then preserves the zero and moves right to the required successor. The full tape is unchanged by these three steps; the final head and mapped control match the old step.

Induction gives, for every old execution length n, a new length m with $n \leq m \leq 3n$ and identical tape/head at the matching boundary. An old halt therefore transfers to a new halt. Conversely, suppose the new machine halts by time n. Simulate n old steps, reaching a matching new time $m \geq n$. Halt is absorbing in the totalized semantics, so the new machine is still halted at m. The checked equivalence $\phi(q) = \text{halt} \iff q = \text{halt}$ forces the old boundary to be halted as well. This proves the reverse implication without another local simulation or any fairness premise. □

The timing lemma is `timed_simulation`; the checked finite endpoint is `checked_fold`. The generic source has 322 lines. The three instances plus their explicit proposition-transfer corollary and audits have 35 lines, with literal generated table data counted separately. This is useful reuse, not a claim that the first generic implementation is shorter than one preexisting hand proof.

3.3 Concrete instances and M125 table

All three instances set

$$(a, b, \text{halt}, u, v, w, r_0) = (124, 127, 254, 62, 14, 138, 8),$$

and choose r_1 as follows:

Instance	Provider r_1	Working states
M124	59	215
FM215	109	215
M125, supplied here	125	215

M125's exact four replacements relative to the primary native table are

Row	Primary K215	M125
110/0	$0R127$	$0R124$
124/0	$0R62$	$0L127$
127/0	$0R14$	$0R8$
127/1	$1R138$	$1R125$

Its native SHA-256 is

`9359aa5ca6dee6db1ec26a32b47d9e95593c6de749ae53f6208ac4ef369256d3.`

The archive includes both native and dense tables, the exact bijective renaming, and their hashes. Each has 215 working controls and 430 rows, with all working controls reachable in the transition graph. Graph reachability is used for this structural census, not as a claim that all controls occur in the blank computation.

The original native labels occupy 303 list slots in the formal data. The unused slots are absent transitions, not working states. The finite audit checks complete binary rows on the working controls and closure of every target into the working controls or the designated halt. Starting from 0 on any tape therefore cannot reach a nonhalt missing-row slot. This relates the totalized formal predicate, which means reaching label 254, to the conventional halting interpretation of these exact machines.

3.4 Fresh verification and precise scope

The generic module, literal tables, and concrete instances compiled from fresh sources. Six printed assumption reports are closed, and recursive `coqchk -silent MarkerInstances` passed. A separate independent review checked the finite checker and every literal row. The no-op-sharing tape update is proved pointwise equal to ordinary tape update; no function extensionality axiom is used.

An independent finite simulator additionally checks each synchronization boundary through the complete calibrated blank prefix. For M124, FM215, and M125 alike, 95,714 old steps correspond to 95,840 new steps, with exactly 63 three-step blocks. Both sides end at native control 84, head -8, and the same complete tape containing 95 ones. These observations corroborate the literal bindings. The arbitrary-tape theorem, not the finite prefix, supplies the unbounded halting conclusion.

The compiled result is a native table-to-table theorem in this packet's arbitrary-tape semantics. A formal bridge to the maintained zipper/dense representation and composition with the full ZFC endpoint remain separate integration tasks. Ordinary mathematical composition with the exact

K215 calibration gives the same ZFC-formalizable consistency equivalence, but that is not advertised as a freshly rebuilt complete assistant endpoint.

A corpus comparison found no rooted state-bijection or global-reflection match for the explicit M125 table among 2,248 tracked `.tm` paths and 87 direct `.tm` members of the seven incoming archives. The qualifying subset contains 640 complete numeric 215-state tables. One lacks an explicit start header and is compared using start 0, as recorded. This is an inventory fact about explicit tables. Because provider 125 was already described, it supplies no discovery-priority claim for the construction.

4 A concrete zero-credit theorem for ZC215

4.1 Why this is a distinct simplification

ZC215 uses DROP tag 0, NIP tag 1, MP tag 2, and AX tags at least 3. Its proposed refill enumerates every raw candidate in order $0, 1, 2, \ldots$. The ordinary proof in the repository gives an injective map from NIP occurrences to DROPs in strictly earlier candidates [5]. The existing GC–ZC abstract prototype assumes the key lowering and injectivity properties. The new `CantorCredit.v` proves those properties for the actual codec, proves total unique decoding, and constructs the literal schedule prefix. It therefore closes a specific formal gap; it does not claim discovery of the injection itself.

In the project's command semantics, DROP pushes one theorem head and NIP removes one. The conservative credit assignment is

$$\mathrm{DROP} = +1, \qquad \mathrm{NIP} = -1, \qquad \mathrm{MP} = \mathrm{AX} = 0.$$

The current AX can add further heads, but the proof below does not need them. That is what makes it useful for future source simplification.

4.2 Exact codec, including zero

Put

$$P(x, y) = \frac{(x + y)(x + y + 1)}{2} + y.$$

The formal file uses the standard-library Cantor pairing with this orientation. Complete command fields are:

Command	Field sequence
DROP	$[0]$
NIP	$[1]$
MP(x)	$[2, x]$
AX$(c; x, y)$	$[c + 3, y, x]$

For fields $a_1, \ldots, a_r$, define

$$E([a_1, \ldots, a_r], z) = P(a_1, P(a_2, \ldots, P(a_r, z) \ldots)).$$

For a nonempty command word L, its raw code is defined backward by

$$\begin{aligned} \rho([k]) &= E(\mathrm{fields}(k), 0), \\ \rho(k :: L) &= E(\mathrm{fields}(k), 1 + \rho(L)) \quad (L \neq []). \end{aligned}$$

In particular $\rho([\text{DROP}]) = 0$. Zero is a valid nonempty raw candidate; a terminal DROP in a longer word is distinguished by the positive continuation at its predecessor. Discarding this zero alias would break the exact scheduler argument.

Lemma 4.1 (Concrete decoding)**.** *Every $n \in \mathbb{N}$ is the code of a unique nonempty command word $W(n)$. The supplied executable decoder satisfies*

$$\rho(W(n)) = n, \qquad W(\rho(L)) = L.$$

Proof. Invert Cantor pairing for the outer tag, then consume precisely the fields prescribed by that tag. This yields a unique command and residual r. For $r = 0$, the word terminates. For $r > 0$, recurse on $r - 1$. The residual is at most the enclosing code, so $r - 1 < n$ in the recursive case. Strong induction proves existence; command-field injectivity and the successor continuation convention prove uniqueness. The concrete fuel-based decoder uses $n + 1$ as a sufficient recursion bound. Its default fuel-exhaustion branch is proved unreachable under that bound. □

No formula-decoding or theoremhood assumption occurs here. The result concerns all raw natural codes, exactly as needed when malformed formula codes or numerical aliases are permitted by the numerical calculus.

4.3 Occurrence injection and prefix safety

Theorem 4.2 (Strict lowering of a selected NIP)**.** *Let command position j of a nonempty word L be NIP. Let L' be obtained by replacing just that command with DROP. Then*

$$\rho(L') < \rho(L),$$

and position j of $W(\rho(L'))$ is DROP.

Proof. The replacement preserves the command arity, all other fields, and every other command. At the changed tag, with residual r, the exact decrease is

$$P(1, r) - P(0, r) = r + 1 > 0.$$

Every surrounding successor and pairing in the earlier command prefix preserves strict inequality. The round-trip decoder theorem supplies the claim about the same position in the lowered word. □

Retaining the position is essential: the objects counted are occurrences, not merely command values or candidate indices. Map a NIP occurrence (L, j) to (L', j). This map is injective, because replacing the image DROP at that same position by NIP recovers L exactly.

Theorem 4.3 (Concrete exhaustive prefix credit)**.** *For every $n, j \in \mathbb{N}$, consider the schedule prefix consisting of*

$$W(0)W(1)\cdots W(n-1) \quad \textit{followed by the first } j \textit{ positions of } W(n).$$

Its number of NIP occurrences is at most its number of DROP occurrences. If the next actual command is NIP, the strict inequality already holds immediately before that command.

Proof. Each included NIP is mapped injectively to a DROP in a strictly smaller raw candidate. That candidate is complete in the same prefix, so all image occurrences are already included. Injectivity proves the weak inequality. Apply it to the prefix including one further NIP: its NIP count increases by one, while its DROP count does not. This yields the strict inequality for the prefix immediately before that NIP. □

The formal weak inequality is `exhaustive_schedule_zero_credit`. The strict version is `exhaustive_credit_before_nip`. They have no abstract scheduling premises. The concrete finite list `schedule_cut n j` contains all positions of the earlier candidates and j positions of the current one. If j exceeds its length, extra positions have no command and contribute neither kind; ordinary execution prefixes use the actual length bound.

4.4 Application and remaining machine work

Represent the current numerical stack as a list of explicit theorem heads above a fixed arbitrary opaque natural tail J. Begin with no explicit heads. DROP increases the protected length, NIP decreases it, and the remaining commands conservatively do not decrease it. The strict part of Theorem 4.3 ensures a protected head exists whenever a NIP is about to execute. Thus it never inspects J. In the actual current ZC proposal, this includes the initialized tail $J = 5$ without asserting that 5 itself encodes a theorem stack [5].

This argument applies to executed commands before the first halt once the machine/source bridge establishes the literal schedule: all earlier candidates have then completed without halting. It cannot justify a physical implementation that skips a candidate or returns to the wrong control. The new file does not model that physical implementation. Its remaining interfaces are exact parsing by the source, exact refill order, command soundness and strong completeness, positive binary simulation, startup, and the inherited numerical-calculus/ZFC calibration.

`CantorCredit.v` has 408 lines. It compiled, printed nine closed assumption reports, and passed recursive `coqchk`. These results make the ZC scheduler argument a reusable concrete theorem. They do not remove the obligations needed for a complete ZC215 endpoint, and they do not reduce the number of physical states. In particular, the present primary K215's zero initialized stack already avoids this arbitrary-tail credit argument; adopting ZC solely to shorten that existing endpoint would not automatically be beneficial.

5 Attempt to remove another state

5.1 What was searched

No 214-state candidate passed the new search. The search covers 21 named native 215-state tables, 155,277 eligible state-pair/table combinations, and 133,827,040 replacement-action checks. All complete pairs failed this particular sufficient local proof test. These numbers do not count distinct complete machines, and failure does not establish different blank behavior.

For old controls $q < r$, neither initially active, identify r with q and map every target accordingly. If the two old rows for a symbol agree after that identification, retain their shared action. Otherwise the backward-history contexts for the two old roles must be disjoint, and the search tries all writes, directions, and retained working targets for the new row. Unlike an identical-row-only screen, both symbol rows may change. When both change, the proof of either row is forbidden to use the other changed row; this makes independently accepted choices compose.

The context overapproximation uses backward graph histories of depth four. A history can stop early at the initial state, leaving the other cells unconstrained. Read guards remove inconsistent histories; writes and moves update the known relative cells. For each compatible history, a symbolic local execution branches on every unknown initial tape bit. Success requires a positive block with exactly the old one-step head displacement, mapped successor control, and whole-tape mutation. Every extra changed cell must be restored.

The bounds are 45 microsteps and 200 explored symbolic nodes per local block. A premature halt, an unsupported transition, or exhaustion of a bound rejects that proof attempt. In particular, exhaustion is not a semantic counterexample. The C++ search is not itself a kernel-verified decision procedure. A positive result would still need an explicit table, certificate, and independent proof before being advertised as a new consistency machine.

Input family	Eligible pairs	Action checks
K215	7,467	6,436,264
commands120	7,434	6,408,016
composed	7,445	6,414,008
marker	7,280	6,271,912
context	7,501	6,463,656
shallow	7,507	6,461,944
direct	7,443	6,414,864
dr215	7,432	6,410,584
forced	7,453	6,421,712
gc-zc	7,131	6,138,376
gc215	7,197	6,195,728
mp-first	7,392	6,372,064
rebalanced	7,381	6,363,504
reencoded	7,541	6,505,600
rho6-120	7,434	6,413,152
rho6-210	7,455	6,431,128
zero-credit	7,372	6,353,232
zero-spare	7,393	6,372,920
zero-spare-mp	7,318	6,308,720
zero-spare-transfer	7,319	6,309,576
zero-spare-z201	7,382	6,360,080
Total	155,277	133,827,040

There were zero surviving complete pairs in every row. The native paths and exact input hashes are retained in `state_search/results.json`. The primary K215 screen alone accounts for 7,467 pairs and 6,436,264 action checks. A pair with no passing first-row action is rejected before the second row is searched; hence multiplying the action count by another row count would misstate the experiment.

5.2 Controls and relationship to earlier searches

The supplied controls accept the known 69/400 marker fold, reject a premature-halt corruption, reject the cyclic 8/127 return provider, and accept a synthetic two-row marker fold. The synthetic example also rejects wrong or unspecified predecessor markers. These exercise the exact risks of confusing an origin selector, permitting a zero-progress cycle, or accepting a block with an unsupported tape premise.

The repository already records a 3,567-pair, 20-step two-row screen in the zero-spare work, and a much larger shared-selector search in DR215. Thus the present contribution is a new implementation and wider table coverage within a documented restricted family, not the invention of two-row folding [7]. The executed source hashes, controls, and counts are supplied so that future runs can be compared without conflating their search spaces.

5.3 What remains plausible

The negative result leaves several substantive possibilities open:

1. **Coordinated replacement rows.** Allow the two changed rows to call each other, and verify their joint positive block instead of proving each independently. A successful certificate must exclude a circular provider such as the failed 8/127 construction.
2. **More extensive tape context.** Search a longer history or a small relational invariant, then check it inductively on all incoming edges. The present records do not determine whether deeper justified contexts, longer local blocks, or coordinated row changes would be the more effective extension.
3. **Source changes that remove a physical role.** Use the concrete zero-credit theorem to decouple scheduler safety from the number of heads added by AX. A modified axiom bundle must still preserve numerical soundness and strong completeness; credit alone does not justify deleting an axiom or its binary implementation.
4. **Joint startup and allocation changes.** Reuse a control only when its two behaviors are separated by a proved tape phase invariant. An initializer that is finished in the blank run can still share controls with the indefinite proof search, but this requires exact entry and return conditions rather than graph reachability alone.

These are proposed directions, not constructions. In particular, the two-marker rule merges two logical roles but reuses the released physical control as the selector. It therefore saves no state. A genuine 214-state result needs to absorb that selector into existing controls, remove another role, or alter a more global encoding. There is no lower-bound conclusion from the present search.

6 Proof architecture and integration plan

6.1 Which proof should be shortened?

There are three different optimization targets. Minimizing the additional proof for a variant favors a short transport from an accepted machine. Minimizing an independent semantic proof favors a simple enumerator and stable command contracts. Minimizing the maintained build favors compact data, efficient checked lookup, and a small affected dependency closure. One candidate need not optimize all three.

Choice	Main advantage	Remaining cost
Primary K215	Both complete endpoints already exist; new source theorem has the exact required type	Adopt checked refinements and rebuild the affected clients
M124 / FM215 / M125	Two incoming-edge invariants; one shared finite checker; arbitrary-tape transfer	Native/dense/zipper bridge and endpoint composition; same inherited calibration
CM215	Very short transfer already delivered; existing concrete dense/Core bridge	One extra origin case; final full-endpoint replay

Choice	Main advantage	Remaining cost
ZC215	Concrete exhaustive scheduler; one occurrence injection handles an arbitrary opaque stack tail	Exact source, binary runtime, startup, and calibration connections
Direct / forced variants	Potential source or allocation freedoms	Derived command contracts, protected-tail accounting, and completeness arguments can be more involved
Rebalanced / reencoded variants	Smaller addresses or changed enumeration parameters	Fewer addresses or lower cutoffs do not alone reduce the number of semantic obligations

The first recommendation is therefore to keep the primary machine while simplifying its proof. For an additional machine, use the two-marker family if the smallest new invariant is the priority, or CM215 if the already supplied representation bridge saves the most integration work. For a substantially independent scheduler proof, develop ZC215 around the concrete theorem in Section 4. These are judgments from the inspected obligations, not measured rankings of complete future proof developments [6, 4].

6.2 Recommended sequence for the current endpoint

1. **Freeze the public interfaces and exact table.** Retain the native and dense table hashes and the public source theorem `p215_dagPerm`. Keep the final statements phrased in terms of the same numerical ZFC-refutation predicate. This lets downstream proofs remain clients of stable statements.

2. **Use the already checked one-sided source argument.** The incoming zero-offset result permits ordinary bisimulation; symmetry supplies the reverse implication. Reuse its compact path codec and exact certificate rather than maintaining offset arithmetic and a second expensive mirrored computation in this instance. The generic offset theory may remain available for other machines.

3. **Adopt the balanced checker behind that interface.** Move the new modules into the maintained namespace, reuse the existing imported dependencies, and replace the body of `p215_dagPerm` by the proved balanced body. Update the certificate producer together with its output so that regeneration preserves the change. Build `Correct215` and `CorrectDense215` under the maintained Rocq toolchain and retain fresh assumption reports.

4. **Combine normalization and balanced storage only after an exact binding proof.** The incoming Lean normalization reduces 2,240 checkpoints to 1,426. Balanced storage and semantic normalization address different costs and can be combined. However, the structural packet's canonical certificate does not pass the unchanged raw-pointer checker. Port the proved decoding or canonical-address correspondence, then prove equality with the appropriate checker specification. This packet has not compiled the combined 1,426-entry balanced instance.

5. **Measure the complete affected build.** Separate generator time, data/exactness compilation, certificate evaluation, and downstream theorem compilation. Report a cold build and an

incremental build with exact reused-object bindings. The current single-run source timing does not predict the total.

The same architecture is suitable for Lean, but the balanced implementation in this delivery is Coq source. It should not be described as a new checked Lean optimization. The incoming Lean ordinary and normalized lanes are the starting point for that port.

6.3 Keep four small semantic interfaces

The clearest maintained mathematical proof separates four statements:

1. **Numerical logic.** A syntactic ZFC refutation exists exactly when the numerical proof system can produce its designated contradiction. Preserve the actual contracts for arbitrary natural codes, aliases, and nondecodable values. Adding an unproved well-formed-formula restriction would change the problem.

2. **Enumeration and commands.** Establish command soundness and strong completeness against the exact candidate order. For ZC215, the closed prefix-credit theorem is the one reusable safety interface. Successful bounded traces cannot replace exhaustive-order or completeness arguments.

3. **Source and binary execution.** Connect semantic steps by positive finite blocks on a quantified tape representation. A generic positive-stuttering theorem discharges halting transport once block contracts and progress are known. Keep exact source/table binding as a finite check.

4. **Initialization.** Prove one exact complete postconfiguration: control, head, every represented field, spare layout, and zero exterior. Structured scan summaries can shorten the finite proof. The incoming semantic seed round-trip and startup-summary proposals are useful candidates for this layer, but their new formal versions were not checked here.

For variants, state these interfaces once and prove only the changed contract. A small instance that reuses a complete parent theorem reduces additional maintenance, while the parent's logical dependency closure remains part of the assurance argument.

6.4 Artifacts and reproduction

The ZIP is a self-contained review packet for the three new formal results. It contains sources and selected successful receipts, with no compiled proof caches or installed toolchain.

Path	Contents
`report.tex`, `sections/`	Complete LaTeX source of this report
`report.pdf`	Rendered report
`balanced_lookup/`	Generic refinement, exact 2,240-entry source instance, inherited minimal dependencies, metrics and receipts
`marker_certificate/`	Generic finite checker, three instances, literal native tables, dense M125 table and renaming, finite audit
`zero_credit/`	Concrete Cantor scheduler proof, candidate comparison, incoming inventory, independent marker review
`state_search/`	Restricted synthesis code, controls, exact counts, input provenance, M125 table inventory
`evidence/`	Independent review of balanced lookup
`manifest.json`	Size and SHA-256 of every distributed file except the manifest itself

From the extracted root, run the following commands separately:

```
python3 verify_packet.py
python3 build_pdf.py
python3 balanced_lookup/reproduce.py --output /tmp/k215-balanced-fresh
python3 balanced_lookup/lookup_metrics.py
python3 marker_certificate/build.py --out /tmp/k215-marker-fresh
python3 marker_certificate/verify.py --output /tmp/k215-marker-finite.json
python3 zero_credit/build.py --out /tmp/k215-credit-fresh
```

The proof commands require Coq 8.18.0 and its standard library; the balanced folder includes its small inherited dependency closure. The generic recursive audit can be added with `-check-generic`; the ordinary baseline can be rebuilt with `-benchmark`. The other two proof drivers include recursive checks. Use fresh output paths to preserve the supplied receipts.

PDF rebuilding requires a standard LaTeX installation with the packages listed in `report.tex`. Python integrity and finite-table checks need no third-party packages. The state search additionally needs a C++17 compiler and the pinned ProveIt checkout:

```
python3 state_search/run_search.py \
  --repo /absolute/path/to/ProveIt --out /tmp/k215-search-fresh
```

`verify_packet.py` checks file hashes and recorded evidence bindings; it is not a replacement for the proof-assistant runs. Logs with zero bytes are expected for successful `coqchk -silent` calls, and their exit statuses are separately recorded. Preserved notices identify inherited construction, semantics, and proposal material.

6.5 Acceptance criteria for a future 214-state claim

A successful next construction should provide the literal complete binary table with at most 214 working states, exact initialization included, and a checked route to the same refutation predicate. A transport route should prove a complete initial-configuration relation, whole-tape preservation at boundaries, positive progress, designated-halt preservation and reflection, and an exact representation bridge. An independent route should discharge the four semantic interfaces above. State labels, graph isomorphism, a long nonhalting trace, or an informal claim that a state is *unused* do not supply those obligations.

The deliverable advances the proof infrastructure and supplies additional checked 215-state instances. The missing one-state reduction remains an open construction problem in this investigation.

References

[1] Vladimir Reshetnikov and contributors. ProveIt: accepted K215 construction, exact table and ordinary proof record. Pinned commit `53269bcab1efc0f82d3022645bd7cf0c22d84959`.

[2] ProveIt. K215 Lean and Rocq verification records; Lean endpoint source; current Rocq source-equivalence proof. Same pinned commit.

[3] ProveIt. Fourth proof-simplification phase: source sizes, dependency selection and incremental verification. Same pinned commit.

[4] ProveIt. Seven incoming proposal archives. All seven were inspected at the same pinned commit. Their exact ZIP names and SHA-256 values, selected paths, and comparison notes are recorded in `zero_credit/incoming_inventory.json` and `zero_credit/comparison.md` in this delivery. The one-sided ordinary Rocq source lane and compact path data used here come from `ZFC215_Proof_Simplification_2026-10-21.zip`.

[5] ProveIt. ZC215 construction; ordinary tag-lowering credit proof; earlier abstract prefix-credit formalization. Same pinned commit. The present concrete formalization credits this existing mathematical injection.

[6] ProveIt. Index of placed K215 alternatives and their verification status. Same pinned commit.

[7] ProveIt. Zero-spare engine search notes; DR215 shared-selector research. Same pinned commit. These precede the restricted search delivered here.